

Gramáticas livres de contexto

Linguagens Formais e Compiladores

Nesta aula estudaremos gramáticas livres de contexto, derivações, árvores sintáticas, ambiguidade e formas normais. O objetivo é compreender como uma linguagem pode ser descrita por regras formais e como essas regras servem de base para a análise sintática em compiladores.

Mapa da aula

GLC

Derivações

Árvores

Ambiguidade

Formas normais

Objetivos de aprendizagem

O que o aluno deve conseguir fazer

Ao final da aula, o estudante deverá identificar os componentes de uma GLC, executar derivações, construir árvores sintáticas, reconhecer situações de ambiguidade e compreender por que as formas normais reorganizam gramáticas sem alterar a linguagem gerada.

Resultados esperados

- 1 definir GLC
- 2 derivar palavras
- 3 desenhar árvores
- 4 avaliar ambiguidade
- 5 normalizar regras

Contexto na hierarquia de Chomsky

Onde entram as GLCs

Na hierarquia de Chomsky, as gramáticas livres de contexto ficam acima das gramáticas regulares em poder expressivo. Elas conseguem representar estruturas que autômatos finitos não reconhecem bem, principalmente quando há dependência aninhada ou necessidade de memória em forma de pilha.



O que é uma gramática livre de contexto?

Definição intuitiva

Uma gramática livre de contexto é um conjunto de regras de substituição. Cada regra troca uma variável por uma sequência de variáveis e terminais. Ela é “livre de contexto” porque a variável pode ser substituída independentemente dos símbolos que estão ao seu redor.

Exemplo de GLC

Exemplo de GLC

$S \rightarrow aSb$

$S \rightarrow \epsilon$

Componentes formais de uma GLC

$G = (V, T, S, P)$

Formalmente, uma GLC é representada por $G = (V, T, S, P)$. V é o conjunto de variáveis, T é o conjunto de terminais, S é o símbolo inicial e P é o conjunto de produções usadas para gerar as palavras da linguagem.

Quatro partes

$G = (V, T, S, P)$

V: variáveis

T: terminais

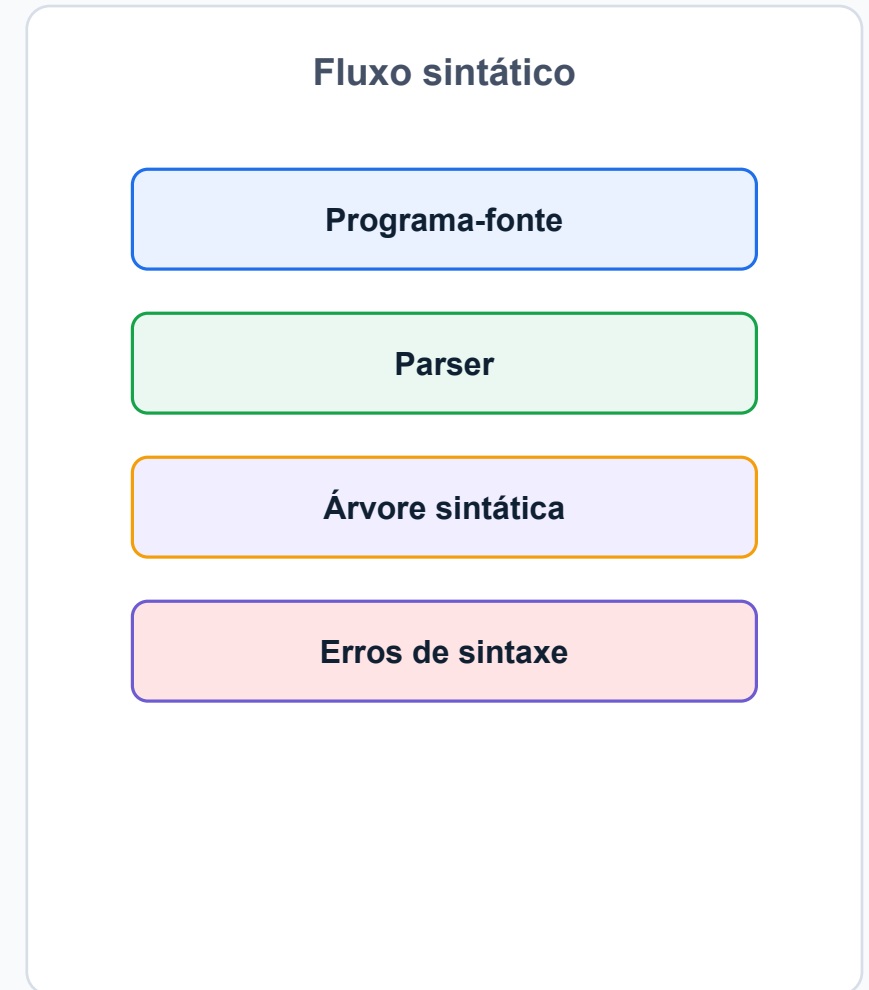
S: inicial

P: produções

Por que GLCs importam em compiladores?

Relação com análise sintática

Compiladores precisam verificar se um programa está escrito conforme a sintaxe da linguagem. Uma GLC pode descrever comandos, expressões, blocos, listas de parâmetros e parênteses balanceados. Por isso, ela está diretamente ligada à construção de analisadores sintáticos.



Linguagem exemplo: $a^n b^n$

Um caso que exige memória

A linguagem $L = \{a^n b^n \mid n \geq 0\}$ contém palavras com a mesma quantidade de símbolos a e b , sempre com todos os a antes dos b . Ela é útil para perceber a diferença entre reconhecer padrão simples e controlar quantidades relacionadas.

Contagem pareada

$$L = \{ a^n b^n \mid n \geq 0 \}$$

ϵ , ab , $aabb$, $aaabbb$, ...

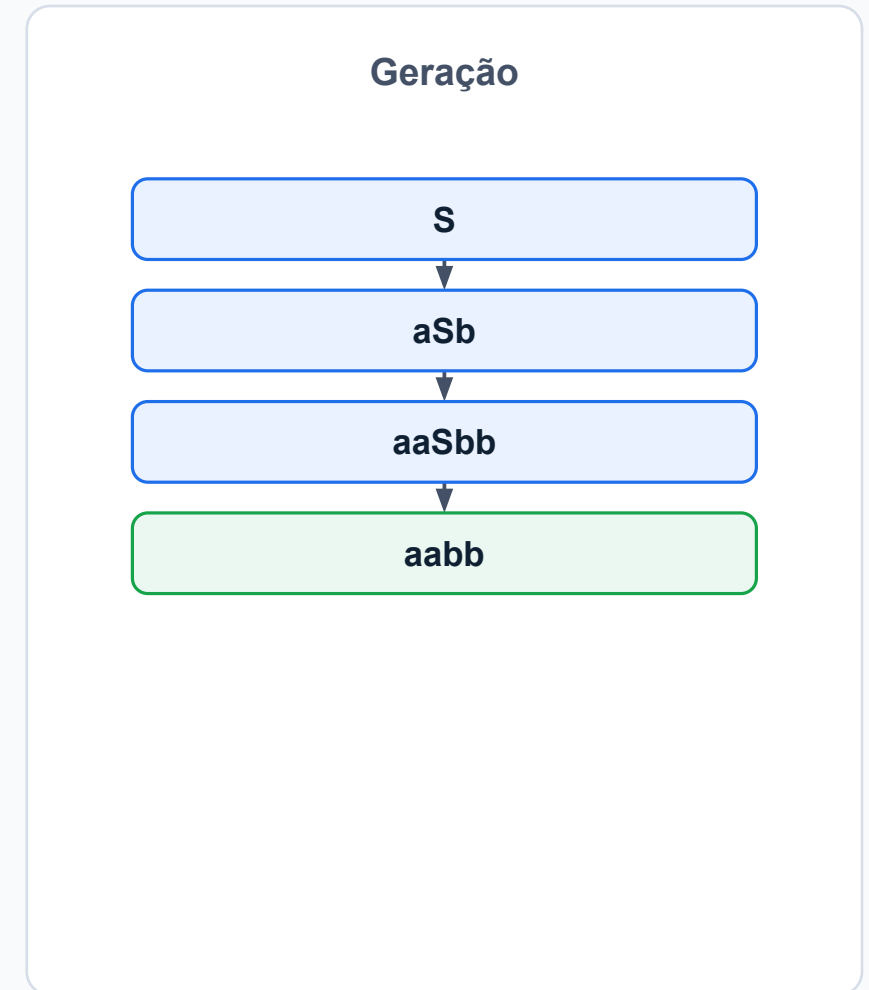
GLC: $S \rightarrow aSb \mid \epsilon$

memória de contagem

Produções e geração de palavras

Como a gramática produz strings

Cada produção funciona como uma instrução de reescrita. A partir do símbolo inicial, aplicamos regras até não restarem variáveis. Quando a sequência final possui apenas terminais, obtemos uma palavra pertencente à linguagem gerada pela gramática.

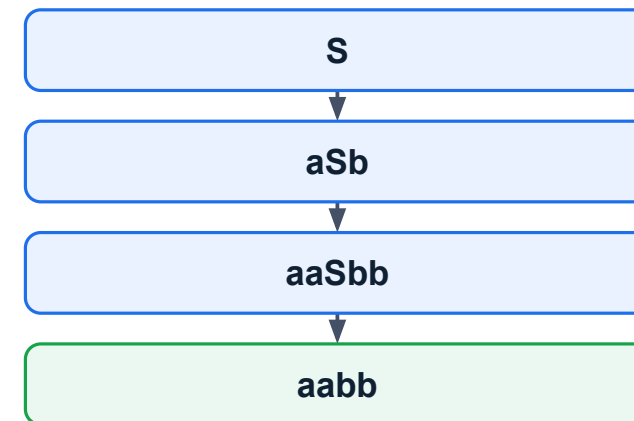


Derivação: conceito central

Sequência de substituições

Uma derivação é o caminho formal que mostra como uma palavra foi construída. Ela registra cada forma intermediária, chamada forma sentencial. Esse registro é importante porque permite justificar que determinada palavra realmente pertence à linguagem definida pela gramática.

Derivação de aabb

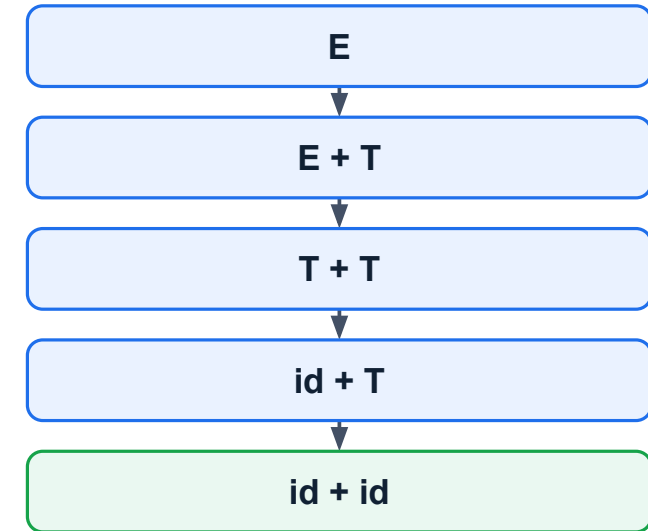


Derivação mais à esquerda

Expandindo a variável mais à esquerda

Na derivação mais à esquerda, sempre escolhemos a variável mais à esquerda da forma sentencial para aplicar a próxima produção. Essa convenção ajuda a padronizar o processo e é muito usada na teoria de parsing e na comparação entre derivações.

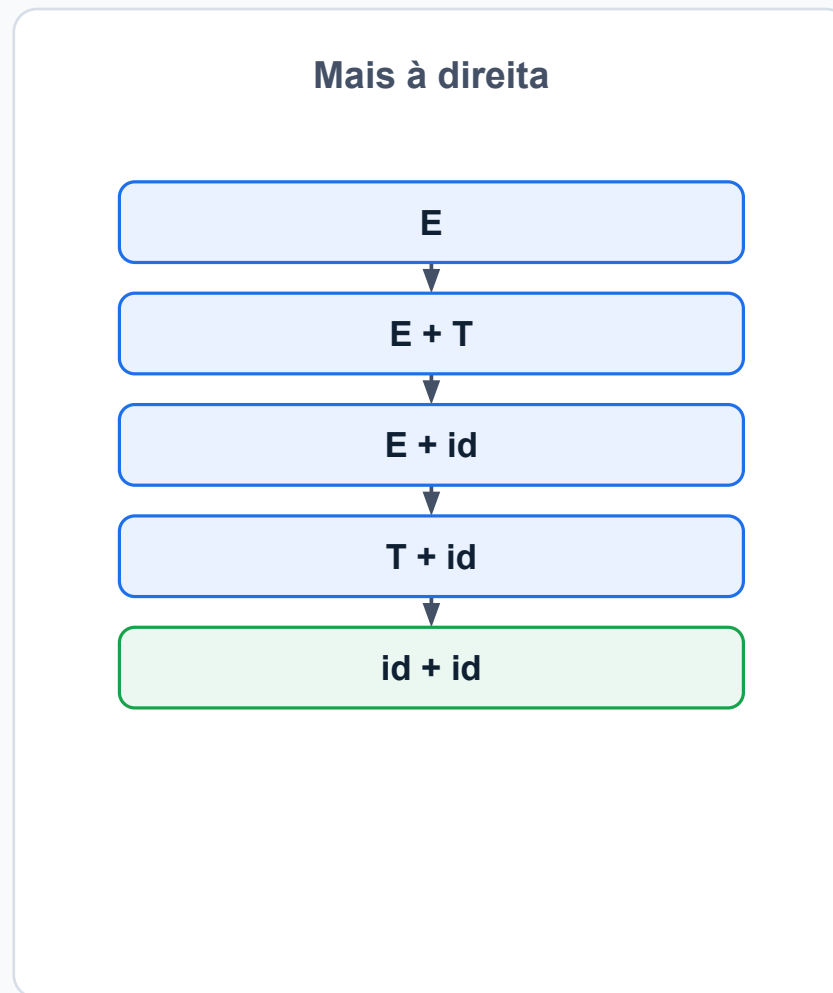
Mais à esquerda



Derivação mais à direita

Expandindo a variável mais à direita

Na derivação mais à direita, a regra escolhida sempre é aplicada à variável mais à direita. Em muitos casos, ela gera a mesma árvore sintática da derivação mais à esquerda, mas o registro da ordem de aplicação das produções fica diferente.



Formas sentenciais

Estados intermediários da derivação

Uma forma sentencial é qualquer sequência obtida durante a derivação, podendo conter variáveis e terminais. Quando a sequência contém apenas terminais, ela deixa de ser intermediária e passa a ser uma sentença ou palavra final da linguagem.

Formas sentenciais

Formas sentenciais

S

aSb

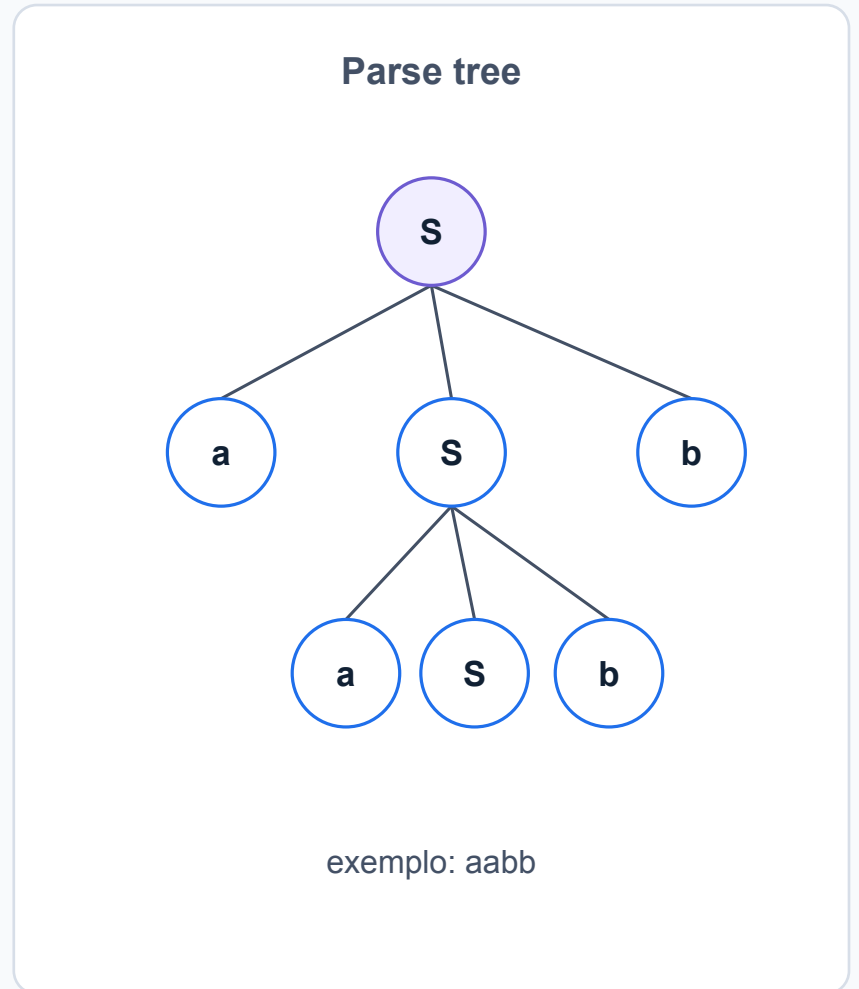
aaSbb

aabb

Árvore sintática

Representação hierárquica

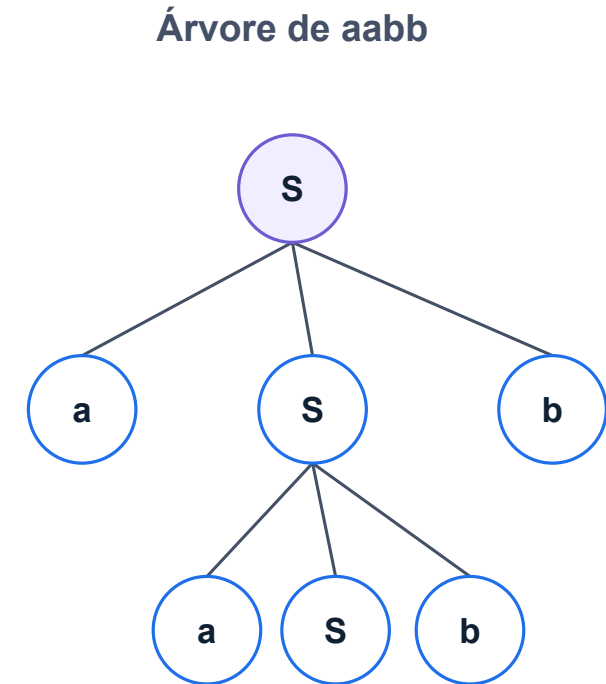
A árvore sintática mostra a estrutura da derivação. A raiz é o símbolo inicial, os nós internos são variáveis e as folhas formam a palavra final. Ela revela a organização interna da sentença, não apenas a ordem das regras aplicadas.



Construindo a árvore de aabb

Exemplo passo a passo

Para construir a árvore, começamos pela raiz S . Aplicamos $S \rightarrow aSb$, criando três filhos: a , S e b . Em seguida aplicamos novamente $S \rightarrow aSb$ no S interno. Por fim, substituímos o último S por ϵ , obtendo as folhas $a a b b$.



exemplo: aabb

Derivação versus árvore sintática

Duas visões do mesmo processo

A derivação enfatiza a ordem de aplicação das produções. A árvore sintática enfatiza a estrutura hierárquica gerada pelas produções. Por isso, duas derivações com ordens diferentes podem representar a mesma árvore, desde que organizem os símbolos da mesma forma.



Parser: o leitor sintático

Da gramática à validação da entrada

Um parser recebe uma sequência de tokens e tenta confirmar se ela pode ser gerada pela gramática. Se conseguir, produz uma árvore sintática ou estrutura equivalente. Se não conseguir, deve indicar erro de sintaxe de forma útil ao programador.

Análise sintática

Programa-fonte

Parser

Árvore sintática

Erros de sintaxe

Gramática de expressões aritméticas

Exemplo de linguagem de programação

Expressões como $\text{id} + \text{id} * \text{id}$ podem ser descritas por uma GLC. Porém, se a gramática for muito genérica, a mesma expressão pode ter interpretações diferentes. Isso mostra por que a escrita das produções precisa considerar precedência e associatividade.

Expressões

Expressões

$E \rightarrow E + E$

$E \rightarrow E * E$

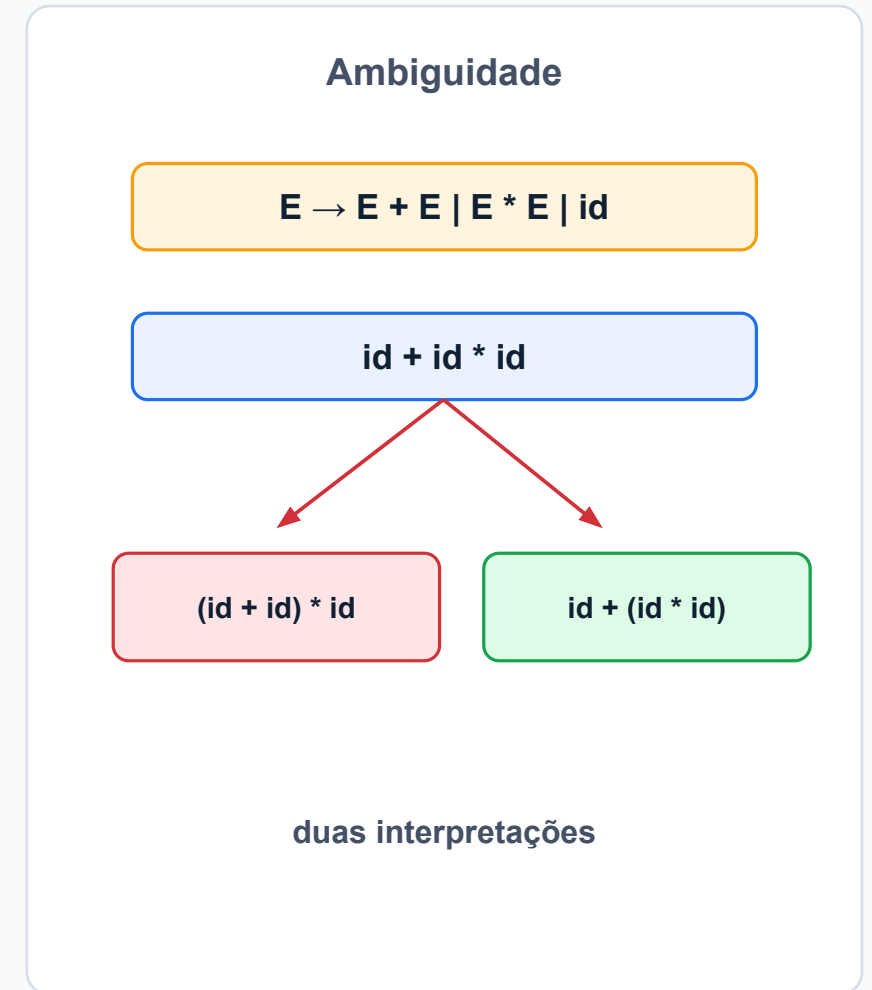
$E \rightarrow (E)$

$E \rightarrow \text{id}$

Ambiguidade em GLCs

Quando há mais de uma interpretação

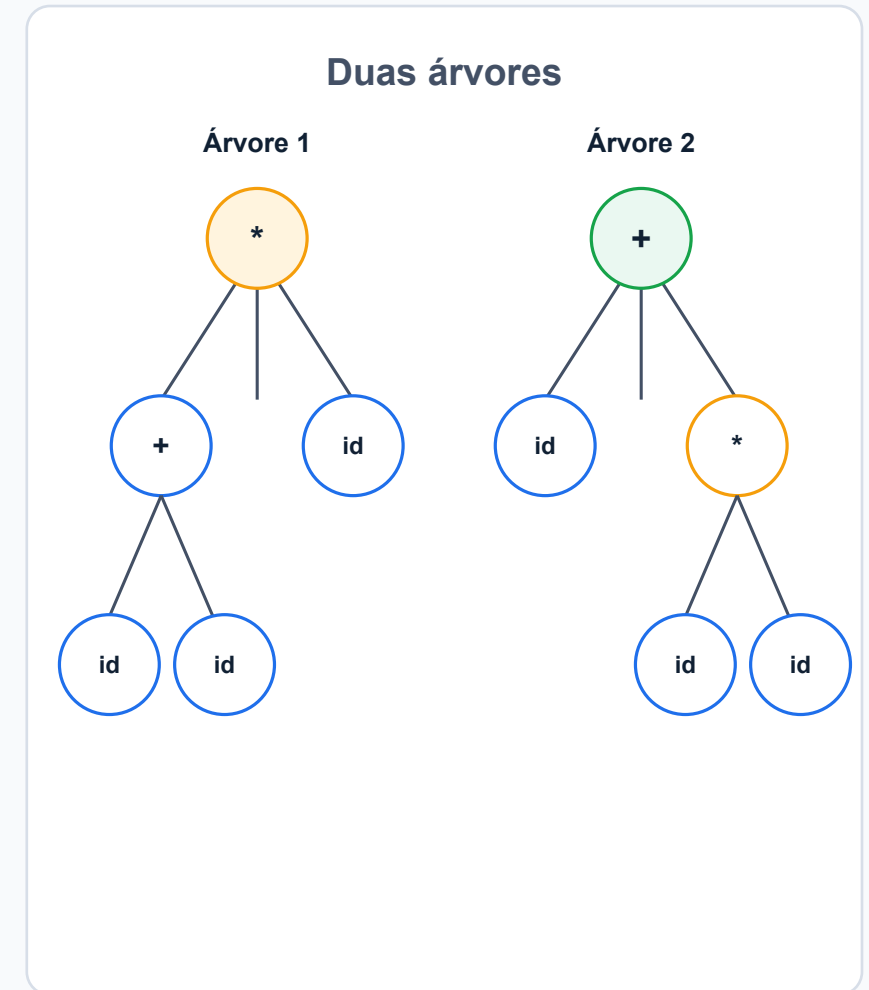
Uma gramática é ambígua quando existe pelo menos uma palavra da linguagem que possui duas árvores sintáticas diferentes, ou duas derivações mais à esquerda distintas. Isso é problemático porque o compilador pode interpretar o mesmo código de formas incompatíveis.



Exemplo ambíguo: $\text{id} + \text{id} * \text{id}$

Duas árvores possíveis

Com a gramática $E \rightarrow E + E \mid E * E \mid \text{id}$, a expressão $\text{id} + \text{id} * \text{id}$ pode ser entendida como $(\text{id} + \text{id}) * \text{id}$ ou como $\text{id} + (\text{id} * \text{id})$. As duas interpretações geram resultados diferentes em linguagens reais.



Impacto da ambiguidade

Por que isso importa na prática

Em uma linguagem de programação, ambiguidade dificulta a implementação do analisador sintático e pode gerar interpretações inesperadas. Por isso, especificações de linguagens costumam eliminar ambiguidade por meio de regras de precedência, associatividade e estruturação cuidadosa da gramática.

Consequências

duas árvores

duas semânticas

erro de projeto

correção da gramática

Reduzindo ambiguidade: precedência

Organizando níveis de operadores

Uma forma comum de evitar ambiguidade é separar operadores em níveis. Soma e multiplicação podem aparecer em variáveis diferentes: E cuida de soma, T cuida de multiplicação e F cuida de identificadores ou expressões entre parênteses.

Precedência

Gramática não ambígua

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow (E) \mid \text{id}$$

* fica em T, + fica em E

Reduzindo ambiguidade: associatividade

Direção do agrupamento

Associatividade define como operadores de mesma precedência são agrupados. Em $E \rightarrow E + T$, a recursão à esquerda sugere associação à esquerda, formando $(id + id) + id$. Isso evita que o parser escolha agrupamentos diferentes para a mesma entrada.

Associatividade

$E \rightarrow E + T$

associa à esquerda

$id + id + id$

$(id + id) + id$

GLCs e linguagens não regulares

Além dos autômatos finitos

Autômatos finitos não possuem memória suficiente para contar quantidades arbitrárias relacionadas. GLCs, em conjunto com autômatos com pilha, conseguem representar padrões como parênteses balanceados e $a^n b^n$, que aparecem em muitas estruturas sintáticas reais.

Memória de pilha

$$L = \{ a^n b^n \mid n \geq 0 \}$$

ϵ , ab, aabb, aaabbb, ...

GLC: $S \rightarrow aSb \mid \epsilon$

memória de contagem

Formas normais: ideia geral

Padronizar sem perder linguagem

Uma forma normal reorganiza as produções de uma gramática em um formato restrito. O objetivo não é mudar a linguagem, mas facilitar provas, algoritmos e análise. Em compiladores e teoria, essa padronização torna certos procedimentos mais previsíveis.

Normalização

gramática original

limpeza

formato restrito

mesma linguagem

Antes da forma normal: limpeza

Simplificações necessárias

Antes de converter uma gramática para uma forma normal, geralmente removemos símbolos inúteis, produções vazias e produções unitárias. Essas etapas reduzem ruído, evitam ciclos desnecessários e deixam a gramática preparada para transformação sistemática.

Etapas comuns

Novo inicial

Remover ϵ

Remover unitárias

Remover inúteis

Binarizar regras

Símbolos inúteis

O que não contribui para a linguagem

Um símbolo inútil é aquele que não ajuda a gerar palavras finais ou que nunca é alcançado a partir do símbolo inicial. Removê-lo simplifica a gramática, porque elimina regras que jamais participarão de uma derivação válida.

Detectar inúteis

- 1 é alcançável?
- 2 gera terminais?
- 3 participa da derivação?
- 4 pode ser removido?

Produções vazias

Regras que geram ϵ

Produções vazias são regras do tipo $A \rightarrow \epsilon$. Elas são úteis para representar opcionalidade, mas podem complicar algoritmos. Ao removê-las, precisamos compensar suas ocorrências nas demais produções para preservar as palavras que antes eram geradas.

Produção vazia

Produção vazia

$A \rightarrow \epsilon$

$S \rightarrow aA b$

gera: ab e aAb

Produções unitárias

Regras do tipo $A \rightarrow B$

Produções unitárias apenas trocam uma variável por outra, sem produzir terminal diretamente. Elas podem criar cadeias longas de substituição que dificultam a análise. Ao removê-las, transferimos para A as produções efetivas que B poderia gerar.

Unitárias

Unitárias

$A \rightarrow B$

$B \rightarrow C$

$C \rightarrow a$

resultado: $A \rightarrow a$

Forma Normal de Chomsky

FNC ou CNF

Na Forma Normal de Chomsky, as produções têm formato muito restrito: uma variável gera duas variáveis, ou uma variável gera um terminal. Opcionalmente, o novo símbolo inicial pode gerar ϵ quando a palavra vazia pertence à linguagem.

FNC

$A \rightarrow B C$

$A \rightarrow a$

$S_0 \rightarrow \epsilon$
(se ϵ pertencer à linguagem)

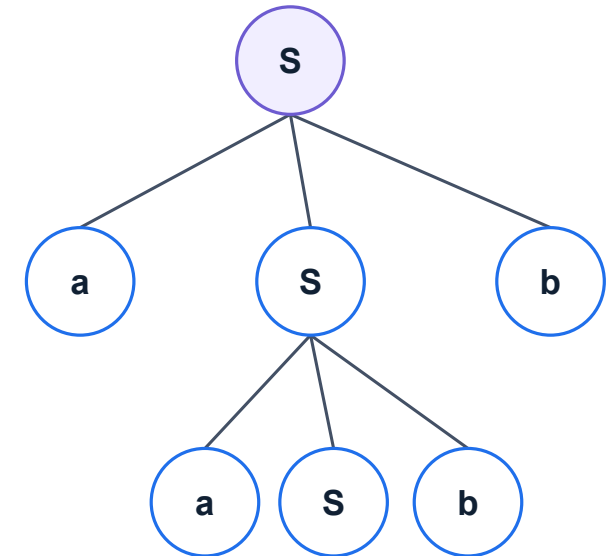
regras curtas, árvore binária

Por que a FNC é útil?

Árvores binárias e algoritmos

A FNC facilita raciocínio formal e implementação de algoritmos de reconhecimento. Como cada produção não terminal se divide em dois símbolos, a árvore sintática fica binária. Isso permite aplicar programação dinâmica em algoritmos como CYK/CKY.

Árvore binária



exemplo: aabb

Conversão para FNC: visão geral

Passos principais

Uma estratégia comum para converter uma GLC para FNC é criar um novo símbolo inicial, remover produções vazias, remover unitárias, eliminar símbolos inúteis, substituir terminais em regras longas e quebrar lados direitos com mais de dois símbolos.

Caminho para FNC

Novo inicial

Remover ϵ

Remover unitárias

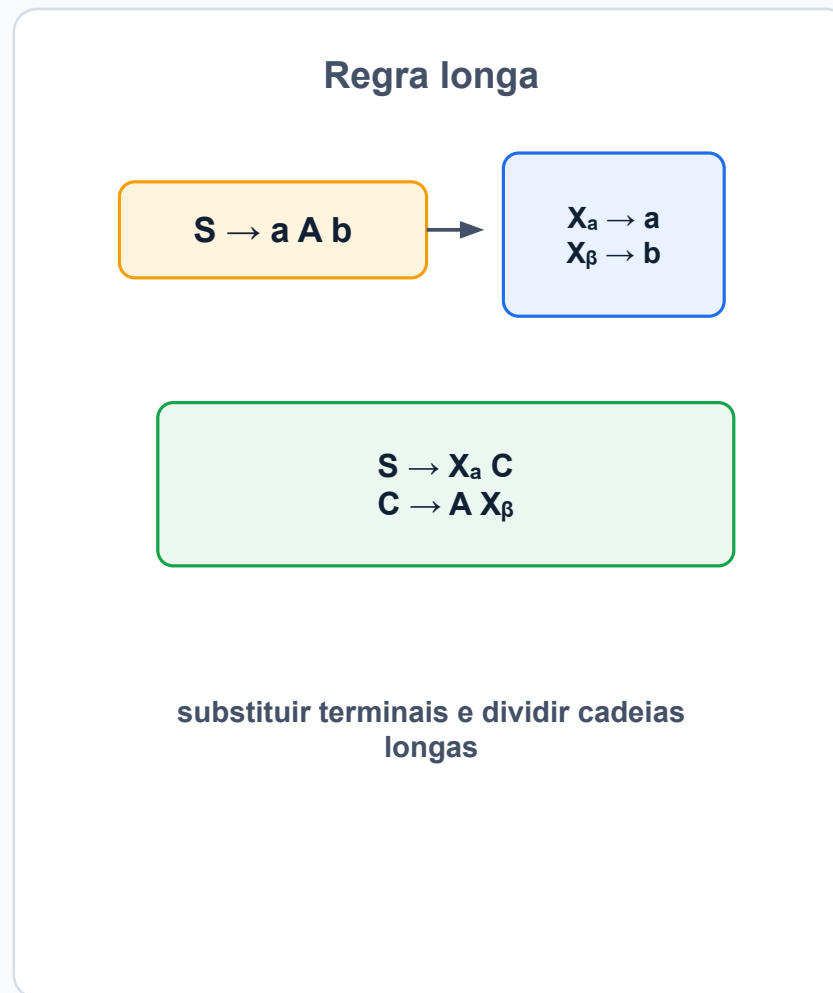
Remover inúteis

Binarizar regras

Exemplo de ajuste para FNC

Transformando uma regra longa

Considere uma regra como $S \rightarrow a A b$. Ela não está na FNC porque mistura terminais com variáveis e possui três símbolos no lado direito. Podemos criar variáveis auxiliares para os terminais e depois dividir a produção em regras binárias.



Forma Normal de Greibach

Outra normalização importante

Na Forma Normal de Greibach, cada produção começa com um terminal, seguido de zero ou mais variáveis. Ela é útil para relacionar derivações ao consumo de símbolos de entrada, embora normalmente seja mais trabalhosa de obter que a Forma Normal de Chomsky.

FNG

$A \rightarrow a \alpha$

começa com terminal

α = sequência de variáveis

FNC versus FNG

Comparação rápida

A FNC é muito usada em provas e algoritmos de parsing com programação dinâmica. A FNG destaca o consumo de terminais no início das produções. As duas são formas normais para GLCs, mas servem a objetivos didáticos e algorítmicos diferentes.

Comparação

FNC: $A \rightarrow BC$

FNC: $A \rightarrow a$

FNG: $A \rightarrow a\alpha$

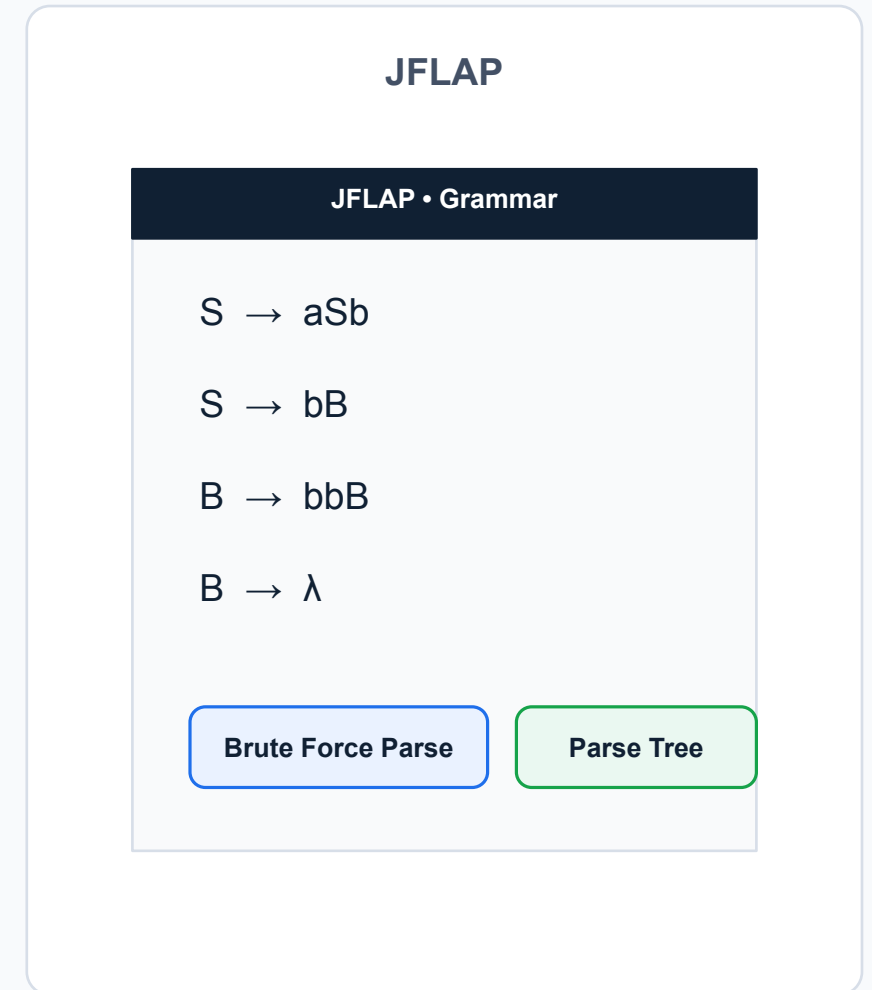
mesma linguagem

Ferramenta prática: JFLAP

Aplicação gratuita acadêmica

O JFLAP é uma ferramenta gratuita usada no ensino de linguagens formais. Ele permite criar gramáticas, testar palavras, visualizar derivações e árvores sintáticas, além de executar transformações como conversão para Forma Normal de Chomsky.

[JFLAP: tutorial de gramáticas](#)



Passo a passo: criar uma GLC no JFLAP

Atividade prática 1

Abra o JFLAP, escolha Grammar e digite uma produção por linha. Para testar a linguagem $a^n b^n$, insira $S \rightarrow aSb$ e $S \rightarrow \epsilon$. Depois use Brute Force Parse para verificar se palavras como ab e $aabb$ são aceitas.

[Tutorial: Brute Force Parser](#)

Entrada da gramática

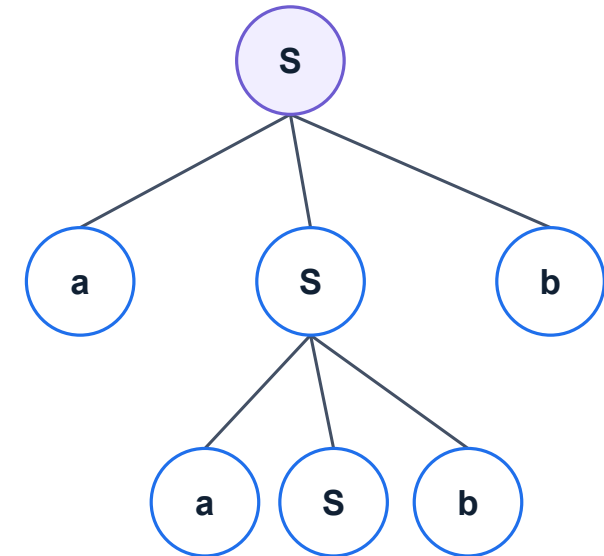
JFLAP • Grammar
$S \rightarrow aSb$
$S \rightarrow bB$
$B \rightarrow bbB$
$B \rightarrow \lambda$

Passo a passo: visualizar árvore sintática

Atividade prática 2

Após executar o parser, alterne a visualização para árvore sintática quando disponível. Observe se as folhas, lidas da esquerda para a direita, formam a palavra testada. Em seguida, compare a árvore com a sequência de derivações exibida pelo programa.

Conferência visual



exemplo: aabb

Passo a passo: testar casos

Atividade prática 3

Monte uma pequena tabela com entradas que devem ser aceitas e rejeitadas. Teste ab, aabb e aaabbb como válidas; teste abb, aab e abab como inválidas. Essa comparação ajuda a descobrir erros na gramática criada.

Tabela de testes

Entrada	Esperado
ab	aceita
aabb	aceita
abb	rejeita
aaabbb	aceita

teste de linguagem

Passo a passo: converter para FNC

Atividade prática 4

No JFLAP, use as opções de transformação de gramática para converter uma GLC para Forma Normal de Chomsky. Acompanhe cada etapa: remover produções vazias, remover unitárias, remover símbolos inúteis e ajustar produções ao formato permitido.

Transformação no JFLAP

Novo inicial

Remover ϵ

Remover unitárias

Remover inúteis

Binarizar regras

[JFLAP: transformação para CNF](#)

Miniatividade em sala

Exercício orientado

Em duplas, criem uma GLC para parênteses balanceados simples, como ϵ , $()$, $(())$ e $(())()$. Depois gerem duas palavras válidas, duas inválidas e desenhem a árvore sintática de uma palavra válida. Finalizem testando a gramática no JFLAP.

Prática em duplas

- 1 definir a gramática
- 2 derivar duas palavras
- 3 desenhar uma árvore
- 4 testar no JFLAP
- 5 registrar conclusões

Erros comuns

Cuidados na resolução de exercícios

Os erros mais comuns são confundir ε com conjunto vazio, usar letras maiúsculas e minúsculas sem critério, esquecer de indicar o símbolo inicial, achar que toda gramática em forma normal é não ambígua e modificar a linguagem durante a simplificação.

Evitar erros

- 1 confundir λ com $\emptyset$
- 2 usar terminal como variável
- 3 mudar a linguagem ao simplificar
- 4 achar que FNC remove ambiguidade

Síntese da aula

O que deve ficar claro

GLCs descrevem linguagens com estrutura hierárquica. Derivações mostram a sequência de regras; árvores sintáticas mostram a organização interna; ambiguidade indica múltiplas interpretações; formas normais padronizam gramáticas para facilitar análise, provas e algoritmos.

Resumo

GLC

Derivações

Árvores

Ambiguidade

Formas normais

Questões para revisão

Verificação de aprendizagem

Explique com suas palavras o que torna uma gramática livre de contexto. Depois derive a palavra aabb usando $S \rightarrow aSb \mid \varepsilon$, desenhe a árvore correspondente e diga por que a gramática $E \rightarrow E+E \mid E^*E \mid id$ é ambígua.

Revisão

- 1 definir GLC
- 2 derivar aabb
- 3 desenhar árvore
- 4 explicar ambiguidade

Referências

Fontes utilizadas e leituras complementares

Rodger, S. H.; Finley, T. W. JFLAP – An Interactive Formal Languages and Automata Package, 2005. JFLAP Tutorials: Brute Force Parser e CFG to CNF. OpenDSA: Context-Free Grammars. Waterloo: Parse trees, ambiguity and CNF. Stanford SLP3: CFG and parsing.

Leituras

JFLAP Book

JFLAP Tutorials

OpenDSA

Waterloo / Stanford